

```

"""
evolution.py
-----
選択・繁殖・突然変異。
ルーレット選択（fitness比例）で多様性を保ちながら進化させる。
"""

import numpy as np
from agent import SoulAgent

def fitness(agent: SoulAgent) -> float:
    """適応度 = H + Z（最低保証あり）。"""
    return max(agent.H + agent.Z, 0.01)

def roulette_select(
    agents: list[SoulAgent],
    n: int,
    rng: np.random.Generator,
) -> list[SoulAgent]:
    """fitness比例のルーレット選択。n個体を非復元抽出。"""
    fitnesses = np.array([fitness(a) for a in agents])
    probs      = fitnesses / fitnesses.sum()
    n_select   = min(n, len(agents))
    indices    = rng.choice(len(agents), size=n_select, replace=False, p=probs)
    return [agents[i] for i in indices]

def reproduce(
    parent: SoulAgent,
    child_id: int,
    child_rng: np.random.Generator,
    mutation_rate: float = 0.05,
) -> SoulAgent:
    """
    親から子を生成する。
    各パラメータに  $\pm \text{mutation\_rate}$  の乗算ノイズをかける。
    Zは70%の確率で親から継承（ $\times 0.8$ ）。
    """
    def mutate(val: float) -> float:
        return val * child_rng.uniform(1 - mutation_rate * 2, 1 + mutation_rate *

    child = SoulAgent(

```

```

        agent_id=child_id,
        rng=child_rng,
        rule_params=(
            mutate(parent.rule_alpha),
            mutate(parent.rule_beta),
            mutate(parent.rule_gamma),
        ),
        stability=mutate(parent.stability),
    )

# テキスト記憶の部分継承（最後の表現だけ渡す）
if parent.text_log:
    child.text_log = [parent.text_log[-1]]

# zの遺伝
if parent.has_Z and child_rng.random() < 0.7:
    child.has_Z = True
    child.Z      = parent.Z * 0.8

return child

def evolve_population(
    agents: list[SoulAgent],
    population_size: int,
    master_rng: np.random.Generator,
    mutation_rate: float = 0.05,
) -> list[SoulAgent]:
    """
    1世代分の選択・繁殖を行い、次世代のエージェントリストを返す。
    """
    survivors = roulette_select(agents, population_size // 2, master_rng)

    next_generation: list[SoulAgent] = []
    next_id = max(a.agent_id for a in agents) + 1

    for parent in survivors:
        child_rng = np.random.default_rng(master_rng.integers(0, 2**31))
        next_generation.append(parent)
        next_generation.append(
            reproduce(parent, next_id, child_rng, mutation_rate)
        )
        next_id += 1

    return next_generation[:population_size]

```

